

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
DEPARTAMENTO DE INGENIERÍA - FABRICUM

DESARROLLO WEB CON PYTHON
EXAMEN FINAL 2025-1

Indicaciones generales:

- Comentar las vistas HTML y las funciones desarrolladas en el servidor
 - Fecha de entrega del proyecto: 26/10/2025 - 23:59 p.m
 - Para un mejor desarrollo del proyecto actualizar su repositorio con su avance diario
 - Compartir sus repositorios con el usuario @af2497
-

Caso de estudio: PROYECTO *miniRedsocial*

A lo largo de las ultimas clases se ha desarrollado la aplicación miniRedsocial en donde se han intergrado diferentes funcionalidades que permiten crear usuarios, subir publicaciones, comentar publicaciones y establecer conversaciones con otros usuarios. Como base del proyecto debe utilizar el archivo miniRedsocial.zip y es dentro de eso proyecto donde se le pide implementar la funcionalidad para poder responder mensajes. Puede verificar el sitio web con la funcionalidad desplegada en <https://afsegovia.pythonanywhere.com/>. Las instrucciones para el desarrollo de la funcionalidad se indican en el detalle de cada pregunta. El proyecto se debe compartir a traves de github con el usuario @af2497. Los usuarios de la aplicación son :

Usuario : Alexander

Contraseña: Alexander

Usuario: Francisco

Contraseña: Francisco

En la función `cargarPub(idPub)` que se encuentra en `comentario.js` se ha agregado un botón con el texto responder y que en su atributo `onclick` llama a la función `mostrarFormularioRespuesta(<id del comentario>,<id de la publicacion>)`. Esa función debe permitir mostrar un formulario basado en un campo de texto y un botón para poder responder al comentario existente. Su tarea es implementar dicho formulario con las indicaciones presentadas:

Una vez implementado el formulario se tendrá la siguiente vista en al cargar una publicación:

Al hacer click en el botón responder gris se debe poder mostrar o ocultar la sección de comentarios (esto se logra con la TOGGLE en la función mostrarFormularioRespuesta). Al hacer click en el botón azul responder se debe llamar a la función enviarRespuesta que recibe como parametros el id del comentario y el id de la publicación. Esta función debe capturar el area de texto en donde se ha escrito la respuesta al comentario, extraer el texto y validar que

tenga caracteres validos (revisar el uso de la función `.trim()` en clases anteriores).

Luego, se debe crear el objeto JSON conteniendo el texto de la respuesta, el id del comentario al cual se esta respondiendo y el id de la publicación padre.

Finalmente se debe ejecutar el envio de la informacion a la ruta `/publicarRespuestaComentario/`. Recordar que es una peticion POST por lo que debe configurar los headers apropiados (revisar las clases 7 y 8 para las peticiones POST).

```
function enviarRespuesta(idComentario, idPublicacion) {  
  // PREGUNTA 1 - FUNCION ENVIAR RESPUESTA HACIA EL BACKEND  
  
  // CAPTURAR EL OBJETO DEL FORMULARIO CREADO PREVIAMENTE EN DONDE SE ESCRIBE LA RESPUESTA AL COMENTARIO  
  
  // VALIDAR QUE EL TEXTO CONTenga CARACTERES VALIDOS USANDO LA FUNCION .TRIM()  
  
  // CREAR EL OBJETO JSON CON LOS DATOS INDICADOS EN EL EXAMEN  
  
  // USAR LA FUNCION FETCH PARA ENVIAR LOS DATOS A LA RUTA '/publicarRespuestaComentario/'  
  
  // AL RECIBIR LA RESPUESTA DEL SERVIDOR LIMPIAR EL AREA DE TEXTO DE RESPUESTA AL COMENTARIO Y LLAMAR A LA FUNCION  
  // cargarPub PASANDO COMO PARAMETRO EL ID DE LA PUBLICACION (idPublicacion)  
}
```

Al recibir la respuesta del servidor debe vaciar el texto del area de respuesta del comentario y llamar a la función `cargarPub(id de la publicación)` para recargar los comentarios y respuestas.

Pregunta 2: Funciones Backend para respuestas comentarios (7 Puntos)

En el archivo `urls.py` ya se encuentra definida la ruta para `/publicarRespuestasComentario/` que llama a la función `publicarRespuestaComentario` en `views.py` de la app1.

```
path('api/enviar/<int:idUsuario>/', views.recibirMensaje, name='recibirMensaje'),
path('api/exportarChat/<int:idUsuario>/', views.exportarChat, name='exportarChat'),

# RUTA PARA PUBLICAR RESPUESTAS A COMENTARIOS
path('publicarRespuestaComentario/', views.publicarRespuestaComentario, name='publicarRespuestaComentario')
]
```

En la función `publicarRespuestaComentario` se debe recibir el objeto json enviado desde el frontend de la aplicación. Los datos provienen de un formulario POST (revisar implementaciones previas) por lo cual se debe cargar la informacion a traves de la funcion `json.loads()`. Esta función debe extraer los datos del id del comentario, id de la publicacion y el contenido de la respuesta, una vez extraidos se debe obtener los objetos correspondientes de la base de datos y crear el nuevo objeto comentario con su respectivo atributo `respuesta_a` con el objeto comentario al cual se esta respondiendo. Finalmente se debe responder con un `JsonResponse` que contenta `{'status':'ok'}`.

```
# PREGUNTA 2
# SI NO DESEA USAR EL CSRF TOKEN EN EL FRONTEND PUEDE UTILIZAR LOS DECORATORS PARA EVITAR SU USO
def publicarRespuestaComentario(request):
    # FUNCION PARA RECIBIR EL COMENTARIO DESDE EL FRONTEND
    if request.method == 'POST' and request.headers.get('X-Requested-With') == 'XMLHttpRequest':
        """
        CARGAR LOS DATOS JSON
        EXTRAER EL CONTENIDO DE LA RESPUESTA, EL ID DEL COMENTARIO PADRE Y DE LA PUBLICACION PADRE
        OBTENER LOS OBJETOS DE LA BASE DE DATOS
        CREAR EL NUEVO OBJETO COMENTARIO CON EL ATRIBUTO RESPUESTA_A CONFIGURADO ADECUADAMENTE
        """

        return JsonResponse({'status': 'ok'})
    return JsonResponse({'error': 'Petición inválida'}, status=400)
```

Finalmente en la función `devolverPublicacion` se debe anexar las respuestas dentro del objeto JSON que se envía como respuesta, esta información se envía a partir del objeto `lista_respuestas`. Como primer paso se deben obtener las respuestas al comentario `comentarioInfo`, luego se deben iterar en todos esos objetos capturados y se debe anexar objetos json en la lista `lista_respuestas` con la información del autor, el contenido o descripción, el id de la respuesta y la fecha.

```
def devolverPublicacion(request):
    idPub = request.GET.get('idPub')
    try:
        objPub = publicacion.objects.get(id=idPub)
        imagen_url = objPub.imagen.url
        comentariosPub = comentario.objects.filter(pubRel=objPub, respuesta_a=None).order_by('fecha_creacion')
        datosComentario = []
        for comentarioInfo in comentariosPub:
            lista_respuestas = []
            # PREGUNTA 2 - CARGAR LAS RESPUESTAS A LOS COMENTARIOS
            # SECCION DEL EXAMEN FINAL PARA AGREGAR LAS RESPUESTAS:

            """
            CREAR EL OBJETO respuestas QUE CONTIENE LOS COMENTARIOS QUE TIENEN EL ATRIBUTO respuesta_a CON
            EL VALOR DEL OBJETO comentarioInfo
            ITERA EN TODAS LAS RESPUESTAS PARA PODER ANEXAR LA INFORMACION EN LA LISTA lista_respuestas
            LOS DATOS A ANEXAR SON EL AUTOR, LA DESCRIPCION O CONTENIDO, EL ID DE LA RESPUESTA Y LA FECHA
            """
            respuestas = comentario.objects.filter(respuesta_a=comentarioInfo).order_by('fecha_creacion')

            for resp in respuestas:
                lista_respuestas.append({
                    'autor': f"{resp.autoCom.first_name} {resp.autoCom.last_name}",
                    'descripcion': resp.descripcion,
                    'id': resp.id,
                    'fecha': resp.fecha_creacion.strftime("%d/%m/%Y %H:%M")
                })
            # FIN SECCION DE AGREGAR LAS RESPUESTAS

            datosComentario.append({
                'autor': f"{comentarioInfo.autoCom.first_name} {comentarioInfo.autoCom.last_name}",
                'descripcion': comentarioInfo.descripcion,
                'id': comentarioInfo.id,

                # NUEVOS VALORES ENVIADOS:
                'fecha': comentarioInfo.fecha_creacion.strftime("%d/%m/%Y %H:%M"),
                'respuestas': lista_respuestas
            })
            # FIN NUEVOS VALORES
        return JsonResponse({
            'titulo': objPub.titulo,
            'autor': f"{objPub.autorPub.first_name} {objPub.autorPub.last_name}",
            'descripcion': objPub.descripcion,
            'imagen': imagen_url,
            'datosComentario': datosComentario
        })
```

Pregunta 3: Mostrar las respuestas en la función cargarPub (3 Puntos)

En la función cargarPub del archivo comentarios.js se debe agregar la sección que permita mostrar las respuestas de los comentarios correspondientes. Iterar a través del atributo respuestas (comenInfo.respuestas) para agregar los bloques HTML que corresponden a las respuestas de cada comentario, definir el bloque bloqueRespuesta el cual debe mostrar la información del autor y el contenido de las respuestas.

```
function cargarPub(idPub)
{
  fetch('/devolverPublicacion/?idPub=${idPub}')
  .then(response => response.json())
  .then(data => {
    console.log(data)
    document.getElementById('tituloPub').innerHTML = data.titulo
    document.getElementById('autorPub').innerHTML = data.autor
    document.getElementById('descripcionPub').innerHTML = data.descripcion
    document.getElementById('idPublicacion').value = idPub
    imgDiv = document.getElementById('imagenPub')
    imgDiv.innerHTML = ''
    imgDiv.innerHTML = `Aun no hay comentarios</p>`
    }

    for (comenInfo of data.datosComentario)
    {
      /*SE AGREGA ESTA SECCION PARA RESPUESTA A COMENTARIOS*/
      bloqueComentario = `
      <div class="border-bottom mb-3 pb-2" id="bloque-${comenInfo.id}">
        <strong>${comenInfo.autor}</strong><br>
        <p>${comenInfo.descripcion}</p>
        <button class="btn btn-sm btn-outline-secondary" onclick="mostrarFormularioRespuesta(${comenInfo.id}, ${idPub})">
          <i class="fa-solid fa-reply"></i> Responder
        </button>
      </div>
      `;

      // PREGUNTA 3 - AGREGAR LAS RESPUESTAS A LOS COMENTARIOS
      /*AGREGANDO RESPUESTAS DE COMENTARIOS */

      // ITERAR EN EL ATRIBUTO comenInfo.respuestas
      // CREAR EL BLOQUE HTML CON LA INFORMACION DE LA RESPUESTA
      // ANEXAR ESE BLOQUE AL OBJETO bloqueComentario

      /* FIN DE RESPUESTAS DE COMENTARIOS */

      seccionComentarios.innerHTML += bloqueComentario
    }
  })
}
```

Entregable: Git/Github (3 Puntos)

Entregar el proyecto con las modificaciones indicadas replicando las funcionalidades presentadas en el sitio web <https://afsegovia.pythonanywhere.com/>. El proyecto debe configurarse como un repositorio de git, debe subirlo a su cuenta persona de github y compartirlo con el usuario "@af2497"

Pueden guiarse del siguiente tutorial para agregar al usuario @af2497 como colaborador en sus repositorios: <https://www.youtube.com/watch?v=86yPaZCF1hk>